

Running LLMs Locally: A Complete Beginner's Landscape

May 2026

Table of Contents

Running LLMs Locally: A Complete Beginner's Landscape	2
How This Guide Is Structured	3
Before You Start (Beginner Checklist)	3
Part 1: Foundations You Actually Need	3
The Inference Pipeline	4
Core Concepts That Drive Day-to-Day Decisions	4
Quantisation	4
GGUF	5
Where Model Files Come From (Before We Go Further)	5
Context Window	5
The KV Cache	5
Temperature and Sampling	6
Part 2: Hardware and Runtime Constraints	6
CPU-Only Inference	6
NVIDIA GPU	6
AMD GPU	7
Apple Silicon (M-series)	7
Multi-Machine and Heterogeneous Setups	7
Part 3: Inference Stack Options	8
llama.cpp and llama-server	8
Ollama	9
LocalAI	10
vLLM	12
LM Studio	13
Jan	13
Open WebUI	13
A Comparison at a Glance	14
Runtime Layer: Why Containers Usually Win	14
Docker vs Podman	14
GPU Access in Containers	15
Part 4: Practical Deployment Blueprints	16
Setup 1: "I want to chat with a model today, minimal effort"	16
Setup 2: "I want a coding assistant in VS Code"	17
Setup 3: "I want a persistent API service with Podman"	17

Setup 4: “I want everything — text, audio, images, agents”	18
Setup 5: “I want to build AI agents”	19
Part 5: Model Selection and Operational Decisions	20
Where Models Live	20
Hugging Face in 90 Seconds (What It Is and Why It Matters)	20
How to Evaluate a Model Page Before Downloading	21
Model Families Worth Knowing (as of May 2026)	21
Sizing Guide	22
Beginner Download Workflow (Low Regret)	22
API Compatibility Layer (Keep This Stable)	22
Retrieval Layer: RAG for Private Knowledge	23
Access Layer: OpenClaw for Remote Personal Assistant Workflows	23
What OpenClaw Actually Does	24
Setting It Up	24
Recommended Models for OpenClaw	25
Running OpenClaw on an Older Laptop: 64 GB RAM, 6 GB VRAM	25
Why You Might Want This Setup	27
Decision Guide by Constraint	27
5-Day Starter Path	28
Conclusion	28
Appendix A: Quick-Command Cheatsheet	29
Appendix B: Further Reading	30

Running LLMs Locally: A Complete Beginner’s Landscape

From Zero to Your Own Private AI Stack, With Containers

~7,800 words · May 2026

You have heard the hype. You have used ChatGPT. Now you want to understand what it would take to run one of these models yourself — on your own hardware, inside your own network, without sending a single token to a third-party server.

This guide is for you.

It is written for someone who is new to both large language models (LLMs) and container technology. You will come away knowing what an LLM inference server is, why containers are the right way to run one, which tools exist across the full spectrum from beginner-friendly to production-grade, and how to make a considered choice based on your hardware and goals.

We will not prescribe a single solution. We will map the territory.

How This Guide Is Structured

To keep this practical, the guide is arranged as one flow:

1. Foundations and concepts you need before touching tooling.
2. Hardware and runtime constraints that determine what is realistic.
3. Inference stack options (engines, servers, and UIs).
4. Deployment blueprints for common goals.
5. Model-selection and operating decisions.

You can read it linearly or jump directly to the part that matches your current bottleneck.

Before You Start (Beginner Checklist)

If you are completely new, sanity-check these basics first:

1. You can run terminal commands comfortably on your OS.
2. You have at least one model target in mind (chat, coding, RAG, agents).
3. You know your hardware numbers: CPU cores, system RAM, and GPU VRAM.
4. You are ready to dedicate disk space (20–100+ GB disappears quickly with multiple models).
5. You are clear on your privacy boundary: local inference does not automatically mean all integrations are private.

If those are true, you are ready.

Part 1: Foundations You Actually Need

Before running anything, you need a mental model of what you are actually running.

A large language model is, at its core, a very large file of numbers — **weights** — that encode a statistical relationship between tokens (roughly, word fragments). When you ask it a question, those weights are loaded into memory and used to predict, one token at a time, what the most plausible next word is given the conversation so far. That process is called **inference**.

The model file for a well-known open model like Meta’s Llama 3.1 8B might be around 4–16 GB depending on how it has been compressed. A 70B parameter model can be 40–80 GB. “Parameters” and “weights” are used almost interchangeably in casual conversation; what matters practically is: more parameters generally means smarter but slower and more memory-hungry.

The Inference Pipeline

When you type a message, the inference pipeline does this:

1. **Tokenise** your input into a sequence of integer IDs
2. **Forward pass** — run those tokens through all the model’s layers, using the weights, to produce a probability distribution over the next token
3. **Sample** from that distribution (or take the most likely token)
4. Append that token to the input and repeat until an end-of-sequence token is produced

All of this is mathematically just matrix multiplication at enormous scale. It is the reason GPUs — which are fundamentally matrix multiplication accelerators — give you dramatically better performance than CPUs alone.

Core Concepts That Drive Day-to-Day Decisions

These terms will come up repeatedly. Getting them straight now will save you a lot of confusion.

Quantisation

Quantisation is the art of representing model weights using fewer bits than they were originally trained with.

Training typically happens in 32-bit or 16-bit floating point. At full 16-bit precision, a 7 billion parameter model needs roughly 14 GB of memory. That is too much for most consumer hardware.

Quantisation reduces the precision of those weights. Common levels you will see:

Notation	Bits per weight	~Memory for 7B model	Quality impact
FP16 / BF16	16	~14 GB	None (training precision)
Q8_0	8	~7 GB	Negligible
Q4_K_M	4	~4.5 GB	Very small
Q2_K	2	~2.5 GB	Noticeable

The **K** in names like Q4_K_M refers to a refinement of the quantisation method (k-quants) that preserves quality better than naive 4-bit by using different precision for different parts of the weight matrix. The _M suffix indicates medium quality within that family. For most

purposes, **Q4_K_M is the sweet spot** — it is small enough to fit comfortably and smart enough to be genuinely useful.

GGUF

GGUF (GPT-Generated Unified Format) is the file format used by llama.cpp and the majority of the local inference ecosystem. You will see `.gguf` files everywhere. It is a single-file format that bundles both the weights and the model’s metadata (vocabulary, architecture, recommended prompt templates) into one portable file.

If you download a model for local inference, you almost certainly want the GGUF version. Look for filenames like `llama-3.1-8b-instruct-Q4_K_M.gguf`.

Where Model Files Come From (Before We Go Further)

Most beginners hit this confusion point early, so let’s make it explicit now.

- **Hugging Face** is the largest public model hub. Think of it as “GitHub for model files”.
- **Ollama Library** is a curated model catalogue that wraps model files in simple names like `llama3.2:3b`.
- **A model page** usually includes: weights, quantised variants, a license, and a model card that explains intended use and limitations.

When this guide says “download from Hugging Face”, it means: open a model page on huggingface.co, choose a compatible file (usually GGUF for llama.cpp-class tools), and verify the license and model card before use. We will go deeper on this in Part 5.

Context Window

The context window is the maximum amount of text the model can “see” at once — both your input and its own previous output. It is measured in tokens. A 4K context window holds roughly 3,000 words. A 128K context window holds about 96,000 words — an entire novel.

Larger context windows consume more memory *during inference* (due to the KV cache, which stores intermediate computations for prior tokens). Running a model at 128K context on a 16 GB GPU requires careful management.

The KV Cache

The KV cache stores the key and value matrices computed for each token in the current context. It grows linearly with context length and is stored in VRAM (or RAM if running

on CPU). This is why “I want to run a 7B model with a 128K context” requires much more memory than “I want to run the same model at 4K context.”

Temperature and Sampling

These are parameters you control at inference time. **Temperature** controls randomness: 0.0 is deterministic (always picks the most likely token), 1.0 is the model’s default sampling behaviour, and values above 1.0 make responses more random and creative (often incoherent at extremes). For code generation and factual Q&A, use low temperature (0.1–0.4). For creative writing, try 0.7–1.0.

Part 2: Hardware and Runtime Constraints

Your hardware determines which models and servers are practical. Here is an honest assessment:

CPU-Only Inference

Every modern laptop and desktop can run quantised LLMs on CPU. The catch is speed.

A modern desktop CPU (e.g., AMD Ryzen 9 7950X with 32 threads) can generate roughly 5–15 tokens per second for a 7B Q4 model. That is readable but feels slow compared to cloud APIs. For a 13B model it drops to 2–5 tok/s. For 70B it becomes impractically slow for interactive use on CPU alone.

CPU inference is legitimate for: automated batch jobs that run overnight, embedding generation (where you are not waiting for output interactively), and small models (1–3B parameters) that actually run fine at 15–30 tok/s on CPU.

llama.cpp in particular is extraordinarily well optimised for CPU inference, using AVX2, AVX-512, and AMX instructions depending on your CPU generation.

NVIDIA GPU

This is the gold standard for local inference. CUDA is the most mature GPU acceleration path.

VRAM	What fits comfortably
6–8 GB	7B model at Q4; 1–3B at full precision

VRAM	What fits comfortably
12–16 GB	13B model at Q4; 7B at Q8
24 GB	34B model at Q4; 13B at Q8
48 GB+	70B model at Q4
2× 24 GB	70B model at Q4 split across both GPUs

The numbers are approximate; the exact fit depends on context length and the specific model architecture.

AMD GPU

AMD support has improved substantially. The software path is ROCm (AMD’s equivalent of CUDA). llama.cpp, vLLM, and LocalAI all support ROCm. The main caution is that ROCm officially supports only a subset of AMD GPU generations (RDNA 2 and RDNA 3 are generally fine; older cards may need workarounds). Performance is competitive with NVIDIA when properly configured.

Apple Silicon (M-series)

Apple Silicon Macs are exceptionally good for local inference — arguably the best consumer hardware per dollar for this use case. The reason is **unified memory**: the CPU, GPU, and Neural Engine all share the same high-bandwidth memory pool, so a 36 GB M3 Max can use all 36 GB for model weights, whereas a 36 GB machine with a discrete GPU would be split between system RAM and VRAM.

An M4 Max with 64 GB unified memory can run a 70B Q4 model and generate 20–30 tokens per second. That is impressive.

llama.cpp uses Apple’s Metal framework for GPU acceleration on Apple Silicon. Ollama and LocalAI both work natively on macOS.

Multi-Machine and Heterogeneous Setups

If you have multiple machines — say, a beefy workstation and a laptop — several tools support distributed inference where a model is split across machines over a network (llama.cpp’s RPC backend, LocalAI’s P2P mode, vLLM’s tensor parallelism). This is an advanced use case but worth knowing exists.

Part 3: Inference Stack Options

This is the heart of the guide. There are many tools, each with a different philosophy. We will cover them honestly.

Command examples in this section may use either Docker or Podman syntax depending on the upstream project docs. In most cases, you can substitute `docker` and `podman` directly.

Before diving into names, one framing helps a lot:

- **Runner/Engine:** does token generation (llama.cpp, vLLM, transformers backend)
- **Server:** exposes an API around an engine (llama-server, Ollama service, LocalAI, vLLM OpenAI server)
- **UI:** gives humans a chat interface (Open WebUI, LM Studio, Jan)

Some products combine two or all three layers. Keeping these layers separate in your head makes the ecosystem much easier to understand.

One more framing question helps narrow choices quickly:

- **If your priority is minimum setup friction:** choose Ollama.
- **If your priority is deep control and tunability:** choose llama.cpp.
- **If your priority is one API for many modalities:** choose LocalAI.
- **If your priority is throughput under concurrent load:** choose vLLM.
- **If your priority is GUI-first exploration:** choose LM Studio or Jan.

llama.cpp and llama-server

What it is: The foundational project that made local LLM inference accessible. Written in pure C/C++ with no heavy framework dependencies. Ported virtually every open model to run efficiently on consumer hardware.

Why it matters: Every other tool in this list either uses llama.cpp internally, was inspired by it, or competes with it. Understanding llama.cpp gives you a mental model for everything else.

The project provides two main tools:

- **llama-cli** — a command-line chat interface. You point it at a GGUF file and start chatting. Instant gratification.
- **llama-server** — an HTTP server exposing an OpenAI-compatible REST API on `localhost:8080`. Any application that knows how to talk to OpenAI's API can talk

to llama-server with no code changes — just point it at `http://localhost:8080/v1` instead of `https://api.openai.com/v1`.

Download a model and start the server

```
llama-server -hf ggml-org/gemma-3-1b-it-GGUF
```

Or point at a file you already have

```
llama-server -m ./llama-3.1-8b-instruct-Q4_K_M.gguf --port 8080
```

The built-in web UI is accessible at `http://localhost:8080` — a basic but functional chat interface.

Backends supported: CUDA (NVIDIA), HIP (AMD), Metal (Apple Silicon), Vulkan (cross-platform GPU), SYCL (Intel), OpenCL, and highly optimised CPU paths. It also supports hybrid inference where a model is partially loaded onto the GPU and the rest runs on CPU — useful if your VRAM is slightly short.

Container support: Official Docker/Podman images are published on GitHub Container Registry (`ghcr.io/ggml-org/llama.cpp`). Different tags exist for different backends (`-cuda`, `-rocm`, `-intel`, etc.).

Ideal for: Developers who want maximum control, anyone building applications that need an OpenAI-compatible endpoint, people who want to understand what is actually happening.

Limitations: No built-in model management or gallery. You manage GGUF files yourself. No user authentication. Single-user oriented by default (though multi-user parallel decoding is supported).

Ollama

What it is: The most beginner-friendly local LLM runner. Ollama wraps llama.cpp (and increasingly other backends) in a polished CLI and local API service with automatic model management.

Why it is popular: The experience is genuinely simple.

Install (Linux)

```
curl -fsSL https://ollama.com/install.sh | sh
```

Pull and run a model

```
ollama run llama3.2
```

Or run in the background and query via API

```
ollama serve
```

```
curl http://localhost:11434/api/generate -d '{"model":"llama3.2","prompt":"Hello"}'
```

Ollama manages a local model registry. `ollama pull`, `ollama list`, `ollama rm` work like Docker image management. Models are stored in a centralised location (`~/.ollama/models` on Linux). The API is available at `localhost:11434` and has both an Ollama-native format and an OpenAI-compatible endpoint at `/v1`.

Container support: Ollama publishes official Docker images. Running with Podman is straightforward.

CPU-only

```
podman run -d --name ollama -p 11434:11434 ollama/ollama
```

NVIDIA GPU

```
podman run -d --name ollama -p 11434:11434 \
  --device nvidia.com/gpu=all \
  ollama/ollama
```

Pull and run a model inside the container

```
podman exec ollama ollama run llama3.2
```

Ideal for: Beginners, anyone who wants a “just works” experience, people building coding assistants or chat UIs who want a quick local backend, anyone integrating with tools like Continue.dev (VS Code extension) or Open WebUI.

Limitations: Less control over model loading parameters than llama.cpp directly. Model library is curated (though you can import custom GGUFs). Not designed for high-throughput multi-user serving.

LocalAI

What it is: The Swiss army knife of local AI. LocalAI is an open-source “AI engine” that aims to be a drop-in replacement for the entire OpenAI API surface — including text, images, audio, embeddings, and video — running locally.

Why it stands out: Instead of wrapping a single inference engine, LocalAI supports **36+**

backends including llama.cpp, vLLM, transformers (Hugging Face), Whisper (speech-to-text), Stable Diffusion (image generation), and more. You get one API surface for all modalities.

CPU-only (simplest)

```
podman run -ti --name local-ai -p 8080:8080 localai/localai:latest
```

NVIDIA GPU (CUDA 13)

```
podman run -ti --name local-ai -p 8080:8080 --gpus all \
    localai/localai:latest-gpu-nvidia-cuda-13
```

AMD GPU

```
podman run -ti --name local-ai -p 8080:8080 \
    --device=/dev/kfd --device=/dev/dri \
    --group-add=video \
    localai/localai:latest-gpu-hipblas
```

LocalAI has a **model gallery** — a curated catalogue you can browse and pull from. It also supports loading models from Hugging Face, Ollama’s OCI registry, or standard OCI registries.

From inside the container or via CLI

```
local-ai run llama-3.2-1b-instruct:q4_k_m
```

```
local-ai run huggingface://TheBloke/phi-2-GGUF/phi-2.Q8_0.gguf
```

```
local-ai run ollama://gemma:2b
```

As of 2026, LocalAI has also added **built-in agent capabilities** — autonomous agents with tool use, RAG (retrieval-augmented generation), and Model Context Protocol (MCP) support — making it an increasingly compelling all-in-one platform.

Ideal for: Users who want a single container to handle text, embeddings, speech, and image generation. Teams who want multi-user support (API key auth, quotas, role-based access). Anyone building a production-grade private AI platform. Kubernetes deployments (Helm chart available).

Limitations: More complex to configure than Ollama. The breadth of features means more moving parts. Backend management (installing/removing backends via OCI images on the fly) is powerful but adds operational surface area.

vLLM

What it is: A high-throughput, production-grade LLM serving engine from UC Berkeley, designed for GPU inference at scale. Not beginner-friendly — but extremely powerful.

Why it is different: vLLM introduced **PagedAttention** — a technique borrowed from operating system virtual memory management — to efficiently manage the KV cache. This means vLLM can serve many simultaneous requests with far better GPU utilisation than naive implementations.

vLLM also features: - **Continuous batching** — new requests are dynamically added to in-progress batches, reducing latency - **Tensor and pipeline parallelism** — split a model across multiple GPUs or machines - **200+ model architectures** supported including all major LLMs, mixture-of-experts models (DeepSeek, Mixtral), and multimodal models - OpenAI-compatible API, plus Anthropic Messages API and gRPC

Install

```
uv pip install vllm
```

Start the server

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-8B-Instruct
```

Or with Podman (Docker syntax is shown in most upstream docs)

```
podman run --gpus all \
  -p 8000:8000 \
  vllm/vllm-openai:latest \
  --model meta-llama/Llama-3.1-8B-Instruct
```

Ideal for: Anyone serving an API to multiple users simultaneously. Production deployments on cloud GPUs. Teams needing high throughput (requests per second). Scenarios where model response latency under load matters.

Limitations: Requires a CUDA-capable NVIDIA GPU (AMD ROCm support exists but is secondary). Not designed for CPU inference. Overkill for personal single-user use. More complex setup than Ollama or LocalAI.

LM Studio

What it is: A polished desktop application for macOS, Windows, and Linux. If Ollama is the developer-friendly CLI approach, LM Studio is the point-and-click GUI approach.

You open it, browse a built-in model catalogue (pulling from Hugging Face), download a model with a progress bar, and start chatting. It also runs a local server on `localhost:1234` with an OpenAI-compatible API.

Ideal for: Non-technical users, people who prefer GUI over CLI, macOS users who want the slickest Apple Silicon experience, anyone who just wants to experiment without touching a terminal.

Limitations: Closed-source (though free). Cannot be run headlessly in a container. Not suitable for server deployments.

Jan

What it is: An open-source (Apache 2.0) desktop application — a “local ChatGPT replacement” — available for macOS, Windows, and Linux (including Flatpak for the Linux desktop). Built with Tauri and React.

Like LM Studio, Jan provides a GUI for downloading and chatting with models. It also runs a local OpenAI-compatible server at `localhost:1337`, supports MCP for agentic capabilities, and can connect to external cloud APIs (OpenAI, Anthropic, Groq) alongside local models.

Ideal for: Users who want an open-source alternative to LM Studio with full transparency into the code. People who want seamless switching between local and cloud models in one interface.

Limitations: Desktop-first, not designed for server/headless use.

Open WebUI

What it is: Not an inference engine but a **browser-based chat interface** that connects to a running backend (Ollama, OpenAI-compatible APIs, or its own built-in llama.cpp engine). Think: a self-hosted ChatGPT-style UI that sits in front of your inference server.

```
# Connect to a running Ollama instance  
podman run -d -p 3000:3000 \
```

```
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \  
ghcr.io/open-webui/open-webui:main
```

Open WebUI supports multi-user accounts, conversation history, RAG via document upload, image generation, and voice input. It is the most popular front-end for self-hosted LLM deployments.

Ideal for: Anyone who wants a proper chat application rather than a bare API. Teams where multiple people need a shared interface. Self-hosters who want the full ChatGPT experience.

A Comparison at a Glance

Tool	Beginner-Friendly	CPU-Only	Container-Native	Multi-User	Throughput	OpenAI API Compatible
llama-server	Medium	Yes	Yes	Partial	Medium	Yes
Ollama	High	Yes	Yes	No	Medium	Yes
LocalAI	Medium	Yes	Yes	Yes	Medium-High	Yes
vLLM	Low	No	Yes	Yes	Very High	Yes
LM Studio	High	Yes	No	No	Low	Yes
Jan	High	Yes	No	No	Low	Yes
Open WebUI	High	N/A (UI only)	Yes	Yes	N/A	Via backend

Runtime Layer: Why Containers Usually Win

LLM servers have version-sensitive dependencies (CUDA/ROCm, Python, compiled backends). Containers isolate that stack so installs are reproducible, easy to replace, and do not pollute your host.

Docker vs Podman

Docker is the established standard. The Docker daemon runs as root in the background, manages images and containers, and most tutorials in the world are written for it.

Podman is a rootless, daemonless alternative that is largely Docker-compatible. It is the default on modern Red Hat/Fedora/RHEL systems and is gaining broad adoption. The key advantages for security-conscious users:

- **Rootless by default** — containers run as your user, not as root. A compromised container cannot escape and damage the host as easily.
- **Daemonless** — no persistent background service. Each `podman` command is a standalone process.
- **Docker compatibility** — most Docker commands work as-is.
- **Podman Desktop** — a GUI application that provides a Docker Desktop-like experience, available on macOS, Windows, and Linux.

For local LLM serving, Docker and Podman are equivalent in most cases. This guide uses `podman`; substitute `docker` if preferred.

GPU Access in Containers

The one area where containers and GPUs require explicit configuration is **device passthrough** — telling the container runtime that it is allowed to see and use your GPU.

NVIDIA with Podman/Docker:

The NVIDIA Container Toolkit installs a hook that makes GPUs available to containers.

Install the NVIDIA Container Toolkit (Fedora/RHEL)

```
dnf install nvidia-container-toolkit
```

Run a container with all GPUs

```
podman run --device nvidia.com/gpu=all ...
```

Or with Docker's syntax (also works with Podman CDI)

```
docker run --gpus all ...
```

AMD with Podman/Docker:

AMD GPUs are exposed as device files in `/dev/kfd` and `/dev/dri`. You mount them directly.

```
podman run \  
  --device=/dev/kfd \  
  --device=/dev/dri \  
  --group-add=video \  
  ...
```

Apple Silicon:

macOS containers (via Podman Desktop's VM, or Docker Desktop) do not have direct Metal GPU passthrough to containers. If you are on Apple Silicon, native installation (without a container) gives you Metal acceleration. You can still use containers on Apple Silicon for CPU-only workloads, or for services that do not need GPU (like Open WebUI connecting to a natively-running Ollama).

Part 4: Practical Deployment Blueprints

Let's translate theory into working setups for the main use cases.

Quick note before you copy commands: image tags, model names, and CLI flags can change over time. If a command fails, check the current official docs for that tool and treat this guide's command blocks as patterns rather than immutable strings.

Setup 1: "I want to chat with a model today, minimal effort"

Use **Ollama**, natively installed or in a container.

Native install (Linux)

```
curl -fsSL https://ollama.com/install.sh | sh
```

Pull a good general-purpose model

llama3.2:3b fits in 4GB RAM/VRAM; llama3.1:8b needs ~5GB

```
ollama run llama3.2:3b
```

You are now in an interactive chat session.

Type /bye to exit, the model stays loaded.

If you want the pretty web UI, add Open WebUI:

```
podman run -d \  
  --name open-webui \  
  -p 3000:3000 \  
  --add-host=host.docker.internal:host-gateway \  
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \  
  -v open-webui:/app/backend/data \  
  ghcr.io/open-webui/open-webui:main
```

Open your browser at `http://localhost:3000`. Create an account (local only), and you have a full ChatGPT-like interface backed by your local Ollama instance.

Setup 2: “I want a coding assistant in VS Code”

The [Continue](#) extension for VS Code connects to any OpenAI-compatible backend. With Ollama running, add this to your `~/.continue/config.json`:

```
{
  "models": [
    {
      "title": "Qwen2.5-Coder 7B (local)",
      "provider": "ollama",
      "model": "qwen2.5-coder:7b"
    }
  ],
  "tabAutocompleteModel": {
    "title": "Qwen2.5-Coder 1.5B (autocomplete)",
    "provider": "ollama",
    "model": "qwen2.5-coder:1.5b"
  }
}
```

After editing the config, reload VS Code so the extension picks up model changes cleanly.

`qwen2.5-coder:7b` is excellent for chat-based coding help. The 1.5B model for autocomplete is fast enough to provide inline suggestions without lag.

For the VS Code extension that turns llama.cpp’s FIM (fill-in-the-middle) capability directly into inline completions, see [llama.vscod](#).

Setup 3: “I want a persistent API service with Podman”

Use a Podman + user-systemd service so your API starts on boot (Linux). On macOS/Windows, use your platform startup tooling.

Start the container

```
podman run -d \
```

```

--name llm-api \
-p 8080:8080 \
--device nvidia.com/gpu=all \
ghcr.io/ggml-org/llama.cpp:server-cuda \
-hf ggml-org/Meta-Llama-3.1-8B-Instruct-GGUF \
--port 8080 \
--host 0.0.0.0

# Generate a systemd unit file
podman generate systemd --name llm-api > ~/.config/systemd/user/llm-api.service

# Enable it
systemctl --user enable --now llm-api

Any app can now call http://your-machine:8080/v1/chat/completions via standard OpenAI SDKs.

```

Setup 4: “I want everything — text, audio, images, agents”

Use **LocalAI** with a persistent volume for models.

`docker run` examples from LocalAI docs usually translate directly to `podman run`.

```

podman run -d \
  --name localai \
  -p 8080:8080 \
  --gpus all \
  -v localai-models:/build/models \
  localai/localai:latest-gpu-nvidia-cuda-13

# Install a text model
podman exec localai local-ai run llama-3.2-3b-instruct:q4_k_m

# Install a speech-to-text model
podman exec localai local-ai run whisper-base

# Now POST to the standard OpenAI endpoints:
# /v1/chat/completions - text generation

```

```
# /v1/audio/transcriptions - speech to text
# /v1/embeddings - embeddings for RAG
```

LocalAI's API is a superset of OpenAI's, so OpenAI-compatible libraries work directly.

Setup 5: "I want to build AI agents"

Agents need reliable **tool calling**: model emits structured tool requests, your code executes them, and feeds results back.

Most modern instruct models support this behind OpenAI-compatible APIs.

Popular agent frameworks that work against any OpenAI-compatible local server:

- **LangChain** / **LangGraph** — the most widely used Python framework for building LLM-powered chains and stateful agents
- **AutoGen** (Microsoft) — multi-agent conversation framework where multiple LLM instances collaborate
- **CrewAI** — high-level abstraction for teams of specialised agents
- **Semantic Kernel** (Microsoft) — .NET and Python SDK for enterprise agent patterns
- **smolagents** (Hugging Face) — lightweight, code-centric agent framework

All of these accept a `base_url` parameter pointing at your local server:

```
from openai import OpenAI

# Point the standard OpenAI client at your local server
client = OpenAI(
    base_url="http://localhost:11434/v1", # Ollama
    api_key="none",                      # no key needed locally
)

response = client.chat.completions.create(
    model="llama3.1:8b",
    messages=[{"role": "user", "content": "What is 2 + 2?"}],
)
```

For agents with tool use, ensure the model you choose has been trained for instruction following and function calling. **Llama 3.1/3.2**, **Qwen 2.5**, **Mistral Nemo**, and **Gemma**

3 are all solid choices. The **instruct** variants (as opposed to base models) are what you want.

Model Context Protocol (MCP) is an emerging standard for exposing tools to LLMs. Both LocalAI and Jan support MCP client-side, meaning you can connect your local LLM to any MCP server (filesystem, databases, web browsers, custom APIs) with standardised tool definitions.

Part 5: Model Selection and Operational Decisions

The server software is only half the equation. You also need to choose which model to run.

If you only remember one rule: **choose model family first, then size, then quantisation**. Many beginners do this in reverse and end up with a model that technically runs but is wrong for their task.

Where Models Live

Hugging Face (huggingface.co) is the primary repository for open models. When looking for models to run locally, filter by the GGUF library. The most reliable quantised GGUF releases come from the model authors themselves or from curators like **bartowski** on Hugging Face.

Hugging Face in 90 Seconds (What It Is and Why It Matters)

If you are new to this ecosystem, Hugging Face can look like “just a download site.” It is much bigger than that.

- It started in 2016 as a conversational AI startup.
- Around 2018–2019, it pivoted hard into open NLP tooling and released the **transformers** library.
- It then became the default collaboration hub for open AI artifacts: models, datasets, evaluation spaces, and demos.

Today, when people say “check Hugging Face,” they usually mean the **Hub**: a Git-based hosting platform for model repositories with version history, model cards, licenses, and community discussion.

For local LLM users, its practical value is:

1. A massive catalogue of open models and quantised variants

2. Transparent metadata (license, architecture, context length, intended use)
3. Reproducibility (pinned revisions, checksums, and clear provenance)

In other words: Hugging Face is not just where files live. It is where the open-model ecosystem publishes, documents, and iterates in public.

Ollama’s model library (ollama.com/library) is a curated subset with clean naming. If you run `ollama pull llama3.2`, Ollama resolves this to the correct GGUF file automatically.

How to Evaluate a Model Page Before Downloading

Do this quick check before pulling large files:

1. **License check:** confirm personal/commercial use rights match your use case.
2. **Model type check:** prefer `instruct`/chat variants for assistants; avoid `base` models unless you know why.
3. **Context check:** verify the maximum context window (important for RAG and agents).
4. **Quantisation check:** start with `Q4_K_M`; move to `Q8` only if quality loss is noticeable.
5. **Compatibility check:** ensure the file format matches your server (`.gguf` for `llama.cpp`/Ollama/LocalAI llama backend).
6. **Provenance check:** prefer trusted publishers and clear conversion notes.

This 60-second pass prevents most “it runs but behaves strangely” problems.

Model Families Worth Knowing (as of May 2026)

Meta’s Llama series — The Llama 3.x models are free for commercial use (with some size restrictions), well-tested, and universally supported. Llama 3.2 3B is a strong small model; Llama 3.1 8B and 70B are the workhorse mid and large options.

Qwen (Alibaba) — Qwen 2.5 and 3 are competitive with Llama at equivalent sizes, with particularly strong coding and multilingual capability. Qwen2.5-Coder is the current go-to for coding-specific tasks. Qwen3 introduces models with a “thinking” mode.

Mistral / Mixtral — Mistral 7B is efficient and capable. Mixtral 8x7B is a Mixture-of-Experts model that is effectively a 47B parameter model that only activates 13B at a time — fast to run relative to its quality.

Gemma (Google) — Gemma 3 models (1B, 4B, 12B, 27B) are available for local use. Strong general capability.

Phi (Microsoft) — The Phi-4 14B model punches above its weight class on reasoning tasks. Very good for the size.

DeepSeek — DeepSeek-R1 and its distillations (including into Llama and Qwen architectures) are strong reasoning models. The distilled versions (e.g., DeepSeek-R1-Distill-Qwen-7B) are practical locally.

Sizing Guide

Your VRAM / Shared Memory	Recommended Model Size
4 GB	3B Q4 (Llama 3.2 3B, Qwen2.5 3B)
6–8 GB	7–8B Q4 (Llama 3.1 8B, Qwen2.5 7B)
12–16 GB	13–14B Q4 (Phi-4 14B, Qwen2.5 14B)
24 GB	32–34B Q4 (Qwen2.5 32B)
48 GB+ or unified	70B Q4 (Llama 3.1 70B)

When in doubt, start with the Q4_K_M quantisation and the model size that fits in your memory with 20% headroom to spare. That headroom goes to the KV cache.

Beginner Download Workflow (Low Regret)

If you are unsure where to start, use this sequence:

1. Pull via Ollama first (fastest success path): `ollama run llama3.2:3b`
2. Validate your use case (chat, coding help, summarisation, RAG prompts)
3. Only then test larger models or direct GGUF downloads from Hugging Face
4. Keep notes on quality vs speed, because model choice is always a trade-off

Treat model selection as benchmarking, not a one-time decision.

API Compatibility Layer (Keep This Stable)

Every tool in this guide speaks (or can speak) the OpenAI Chat Completions API. This is important because:

1. **Every AI library in every language** has been written to target OpenAI. You do not need to learn a new SDK. You change a URL and, often, remove the API key.
2. **You can swap backends without rewriting code.** If you start with Ollama and later need vLLM's throughput, the change is a configuration line.

3. **Commercial and local models become interchangeable.** You can build an application that uses your local Llama instance during development and switches to GPT-4o in production — same code.

The key endpoints:

Endpoint	Purpose
POST /v1/chat/completions	Multi-turn conversation (the main one)
POST /v1/completions	Raw text completion (older format)
POST /v1/embeddings	Generate embedding vectors for RAG
GET /v1/models	List available models

Retrieval Layer: RAG for Private Knowledge

A base LLM has a training cutoff and no knowledge of your private documents. **Retrieval-Augmented Generation (RAG)** solves this by:

1. Converting your documents into numerical vectors (**embeddings**) using an embedding model
2. Storing those vectors in a vector database (Chroma, Qdrant, pgvector)
3. At query time, converting the question into a vector, finding the most similar document chunks, and injecting them into the LLM's context window as additional context

For local RAG you need two models running: the LLM for generation and an embedding model for the retrieval step. Embedding models are much smaller (50–500 MB) and fast on CPU.

Good local embedding models: `nomic-embed-text` (via Ollama), `all-minilm-l6-v2` (via LocalAI/llama-server), `mxbai-embed-large`.

Open WebUI has RAG built in — you can upload documents directly in the chat interface and it handles the embedding and retrieval transparently.

Access Layer: OpenClaw for Remote Personal Assistant Workflows

OpenClaw is the “last-mile” gateway: your model runs locally, but you can interact through messaging channels (WhatsApp, Telegram, Slack, Discord, iMessage).

What OpenClaw Actually Does

OpenClaw runs a persistent local gateway. Messages are relayed from chat apps to your local model backend and returned as replies:

```
Your phone -> messaging service -> OpenClaw gateway (your machine) -> Ollama -> model
|
Your phone <- messaging service <- OpenClaw gateway <----- response
```

It also supports coding-agent workflows, bundled web search, MCP tools, and a local TUI.

Important boundary: message transport still uses each messaging provider's infrastructure, but model execution and assistant state remain local.

Setting It Up

OpenClaw is launched directly through Ollama:

```
# Ollama handles installation, model selection, and daemon startup
ollama launch openclaw
```

On first run, Ollama walks you through: 1. Installing OpenClaw via npm (if not already present) 2. A security notice explaining what tool-level access the agent will have 3. Model selection — local or cloud 4. Configuring your messaging provider(s) and starting the gateway daemon

To connect your messaging apps:

```
openclaw configure --section channels
```

This opens an interactive setup for WhatsApp, Telegram, Slack, Discord, or iMessage. Each provider has its own connection mechanism (QR code scan for WhatsApp, bot token for Telegram, etc.).

To launch headlessly — useful if you want to start it automatically at boot or inside a container:

```
# --yes skips interactive prompts; --model is required
ollama launch openclaw --model qwen3.5 --yes
```

To stop the gateway:

```
openclaw gateway stop
```

Recommended Models for OpenClaw

OpenClaw is an agentic assistant — it does multi-turn reasoning, uses tools, and processes long context. The official documentation recommends **at least a 64K token context window** for local models. This is because agent loops accumulate tool call results, conversation history, and web search output into the context very quickly.

Model	VRAM needed	Notes
qwen3.5 (local)	~11 GB	Reasoning, coding, vision — the local sweet spot
gemma4 (local)	~16 GB	Strong reasoning and code
qwen3.5:cloud	None local	Falls back to Ollama cloud; good for testing
kimi-k2.5:cloud	None local	Multimodal reasoning with sub-agents

Running OpenClaw on an Older Laptop: 64 GB RAM, 6 GB VRAM

6 GB VRAM cannot fully host recommended OpenClaw models, but 64 GB RAM is enough for strong partial-offload setups.

Use **partial GPU offloading**: put as many layers as possible on GPU and keep the rest on CPU/RAM.

llama.cpp's `--n-gpu-layers` flag controls how many layers go to the GPU. A 7B model has 32 transformer layers; a 13B has 40.

Practical configuration for a 7B model on 6 GB VRAM:

```
# With llama-server: load 28 of 32 layers onto the 6GB GPU, rest on CPU
llama-server -m ./qwen2.5-7b-instruct-Q4_K_M.gguf \
  --n-gpu-layers 28 \
  --ctx-size 65536 \
  --port 8080
```

For a 7B Q4 model (~4.5 GB), `--n-gpu-layers 28` typically uses ~3.5-4 GB VRAM, leaving KV-cache headroom. Remaining layers run on CPU. 64 GB RAM is enough for large context KV cache.

With Ollama, the equivalent is set via an environment variable or a Modelfile:

Set GPU layer count globally for Ollama

```
OLLAMA_NUM_GPU=28 ollama serve
```

Or create a custom Modelfile for a specific model

```
cat > Modelfile << 'EOF'
```

```
FROM qwen2.5:7b
```

```
PARAMETER num_gpu 28
```

```
PARAMETER num_ctx 65536
```

```
EOF
```

```
ollama create my-qwen-laptop -f Modelfile
```

```
ollama run my-qwen-laptop
```

What to expect performance-wise:

On modern Intel/AMD laptops with partial offload, 7B Q4 typically delivers ~8-15 tok/s. Long-context prefill is slower, but interactive chat is usable.

Your advantage is context headroom. 64 GB RAM absorbs large KV cache growth during long agent loops.

A concrete setup for this laptop:

1. Install Ollama natively (not in a container - container GPU passthrough

on laptops with integrated+discrete GPU can be tricky)

```
curl -fsSL https://ollama.com/install.sh | sh
```

2. Create a Modelfile tuned for your laptop

```
cat > ~/qwen-laptop.Modelfile << 'EOF'
```

```
FROM qwen2.5:7b-instruct-q4_K_M
```

```
PARAMETER num_gpu 28
```

```
PARAMETER num_ctx 65536
```

```
PARAMETER num_thread 8
```

```
EOF
```

```
ollama create qwen-laptop -f ~/qwen-laptop.Modelfile
```

3. Launch OpenClaw against this model

```
ollama launch openclaw --model qwen-laptop
```

`num_thread 8` is a starting point. Tune toward physical core count; too many threads can add overhead.

Why You Might Want This Setup

The main benefit is not raw speed. It is **privacy + persistence** with local files, databases, and MCP-connected tools.

For a 64 GB RAM / 6 GB VRAM laptop, this enables a practical roaming assistant with large retained context.

Decision Guide by Constraint

Here is how to cut through the options:

Start here: Are you on macOS with Apple Silicon? → Install Ollama natively. Skip the container complexity for now. Add Open WebUI if you want a proper UI.

Are you a Linux user with an NVIDIA GPU who wants to experiment? → Ollama in a Podman container for ease, or llama-server directly if you want to understand what is happening.

Do you want a single container that handles text + embeddings + speech? → LocalAI. Accept that it is more complex to configure.

Are you building an application or API that multiple users will hit? → vLLM if you have an NVIDIA GPU and need throughput. LocalAI if you need CPU compatibility or the multimodal features.

Do you want the least possible complexity, GUI-first? → LM Studio or Jan (Jan if you want open source).

Do you want to build agents? → Any of the above for the inference backend. Add LangChain, AutoGen, or CrewAI on top. The LLM is infrastructure; the agent logic is your application code.

Do you want everything on a home server accessible to your whole household? → LocalAI or Ollama + Open WebUI in Podman, with user accounts in Open WebUI. LocalAI has native multi-user support; Open WebUI provides it as a layer on top of Ollama.

Do you want to chat with your local model from your phone or messaging apps? → OpenClaw via `ollama launch openclaw`. If your machine has plenty of RAM but modest

VRAM (e.g., 6 GB GPU, 64 GB RAM), use partial GPU offloading with a 7B model — you get a responsive assistant with a large context window at no cloud cost.

5-Day Starter Path

Rather than getting lost in choices, here is a concrete sequence for a beginner:

Day 1 — Get something running in 15 minutes:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama run llama3.2:3b
```

Chat with it. See what it can and cannot do. Appreciate the speed difference with and without GPU.

Day 2 — Add a web UI: Stand up Open WebUI pointing at your Ollama. Create an account. Upload a document. Try RAG.

Day 3 — Try the API:

```
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama3.2:3b",
    "messages": [{"role": "user", "content": "Explain containers in one sentence."}]
  }'
```

Now you understand why everything in this ecosystem being OpenAI-compatible is powerful.

Day 4 — Containerise it properly: Move Ollama into a Podman container with a persistent volume for models and a systemd user service so it survives reboots.

Day 5 — Explore the model landscape: Try `qwen2.5-coder:7b` for coding tasks. Try `omic-embed-text` for embeddings. Pull a larger model and compare quality.

From there, the path branches based on what you are building.

Conclusion

The local LLM ecosystem has matured faster than almost any other area of software in recent memory. Two years ago, running a capable model locally required navigating undocumented

build systems and arcane CUDA dependencies. Today, `ollama run llama3.2` gets you there in thirty seconds.

The core insight to take away is this: **the model and the server are separate concerns**. The model is a GGUF file — a portable, version-controlled artefact you can move between machines and servers. The server is infrastructure — swap it for a faster one, add a GPU, put it behind a load balancer — without touching your application code, because they all speak the same OpenAI-compatible API.

That API compatibility is the real unlock. It means the entire ecosystem of LLM tooling — agent frameworks, RAG pipelines, evaluation harnesses, chat interfaces — works against your private local stack with a one-line configuration change.

You own the weights. You own the compute. Nothing you type leaves your machine.

Appendix A: Quick-Command Cheatsheet

=== Ollama ===

```
ollama pull llama3.1:8b          # Download a model
ollama run llama3.1:8b          # Interactive chat
ollama list                     # List downloaded models
ollama rm llama3.1:8b          # Remove a model
ollama serve                    # Start API server (port 11434)
```

=== llama-server ===

```
llama-server -hf ggml-org/gemma-3-1b-it-GGUF      # Download from HF and serve
llama-server -m model.gguf --port 8080 --n-gpu-layers 99 # Load to GPU
```

=== Podman + Ollama ===

```
podman run -d --name ollama -p 11434:11434 \
  -v ollama-data:/root/.ollama ollama/ollama
podman exec ollama ollama pull llama3.2:3b
```

=== Podman + Open WebUI ===

```
podman run -d --name open-webui -p 3000:3000 \
  --add-host=host.docker.internal:host-gateway \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
```

```
ghcr.io/open-webui/open-webui:main

# === Podman + LocalAI (NVIDIA) ===
podman run -d --name localai -p 8080:8080 \
  --device nvidia.com/gpu=all \
  -v localai-models:/build/models \
  localai/localai:latest-gpu-nvidia-cuda-13

# === Python: Use any local server with OpenAI SDK ===
# pip install openai
python3 -c "
from openai import OpenAI
client = OpenAI(base_url='http://localhost:11434/v1', api_key='none')
r = client.chat.completions.create(
    model='llama3.2:3b',
    messages=[{'role': 'user', 'content': 'Hello!'}]
)
print(r.choices[0].message.content)
```

Appendix B: Further Reading

- [llama.cpp README](#) — the engine under most of this ecosystem
- [Ollama documentation](#) — model library, API reference, integrations
- [LocalAI documentation](#) — full backend list, GPU acceleration guide
- [vLLM documentation](#) — production serving, PagedAttention explainer
- [Open WebUI](#) — the most popular self-hosted chat UI
- [Hugging Face GGUF models](#) — browse the model catalogue
- [LM Studio](#) — GUI desktop app
- [Jan](#) — open-source desktop app